

COM S / S E 4170

Software Testing

Spring 2026

Using a Code Coverage Tool - Demo

1

JaCoCo

- One of many code coverage tools for Java
- <https://www.jacoco.org/>

JaCoCo - Java Code Coverage Library

JaCoCo is a free Java code coverage library distributed under the [Eclipse Public License](#). Check (<http://www.jacoco.org/jacoco>) for updates and feedback. This is the distribution of version 0.8.8.202201031026 created on 2022/01/03 based on commit [f046ba52ef6128b0f12151013d9019d71946954](#).

Contents

- [Documentation](#)
- [JUnit Test Results](#)
- [Code Coverage Report \(CSV, XML\)](#)
- [Change History](#)
- [License](#)

The JaCoCo distribution contains the following libraries in the ./lib folder:

File	OSGi Bundle	Description	Dependencies
jacocoagent.jar	no	Java agent for execution data recording	- (all dependencies included)
jacococli.jar	no	JaCoCo Ant tasks	Ant (all other dependencies included)
jacoco.jar	no	JaCoCo Command Line Interface	- (all dependencies included)
org.jacoco.agent-0.8.8.202201031026.jar	yes	JaCoCo agent	-
org.jacoco.core-0.8.8.202201031026.jar	yes	JaCoCo core	ASM
org.jacoco.report-0.8.8.202201031026.jar	yes	JaCoCo reporting	org.jacoco.core, ASM
org.jacoco.ant-0.8.8.202201031026.jar	yes	JaCoCo Ant tasks	org.jacoco.core, org.jacoco.agent, org.jacoco.report, ASM, Ant

All libraries as well as the the JaCoCo [Maven plug-in](#) are also available from the [Maven repository](#).

2

JaCoCo

- Uses **JUnit** for testing
- **Instruments** classes to provide code coverage
- Get both a **test report** and **code coverage report**

3

Other Similar Tools

- Cobertura, Emma, CodeCover, OpenCover..
- For python there is coverage.py

4

JUnit

```

Running TriangleTypeTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.106 sec

Results :

Tests run: 11, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.292 s
[INFO] Finished at: 2022-01-25T02:55:56-06:00
[INFO] -----
mcohen>

```

5

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cov	Missed	Lines	Missed	Methods	Missed	Classes
TriangleType	90%	86%	4	13	1	10	1	2	0	1	0	1
Triangle	100%	n/a	0	1	0	2	0	1	0	1	0	1
Total	3 of 93	96%	3 of 22	86%	4	14	1	12	1	3	0	2

```

4. //Chapter 6:
5. // See TriangleTypeTest for JUnit tests
6.
7.
8.
9.
10. public class TriangleType
11. {
12.     //
13.     * @param s1, s2, s3: sides of the putative triangle
14.     * return enum describing type of triangle
15.     *
16.     public static Triangle triangle (int s1, int s2, int s3)
17.     {
18.         // Reject non-positive sides
19.         if (s1 <= 0 || s2 <= 0 || s3 <= 0)
20.             return Triangle.INVALID;
21.
22.         // Check triangle inequality
23.         if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
24.             return Triangle.INVALID;
25.
26.         // Identify equilateral triangles
27.         if ((s1 == s2) && (s2 == s3))
28.             return Triangle.EQUILATERAL;
29.
30.         // Identify isosceles triangles
31.         // original program line is below (correct version)
32.         if ((s1 == s2) || (s2 == s3) || (s1 == s3))
33.             // fault introduced below by M.Cohen
34.             // if ((s1 == s3) || (s2 == s3) || (s1 == s2))
35.             return Triangle.ISOSCELES;
36.
37.         return Triangle.SCALENE;
38.     }
39. }
40.

```

6

In This Class

- We will use JaCoco to explore the idea of code coverage
- I will provide a package on Canvas that runs on Pyrite. It may also run on other machines. You can also try to get the source from Github yourself
- We will use Apache Maven (<https://maven.apache.org/>) to build and run the tests. I will provide a pom.xml file that is set up for the required tasks

7

Can run from Eclipse

Java Code Coverage for Eclipse

Install

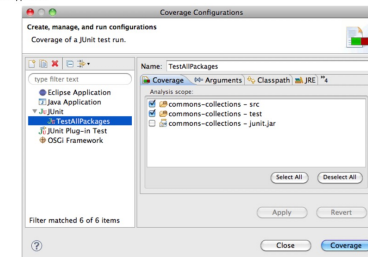
Launching in Coverage Mode

Eclipse allows running Java programs directly from the workbench. Programs can be launched in different so-called launch modes. In a standard Eclipse installation you can launch your programs either in Run or in Debug mode. EclEmma adds a new launch mode Coverage which is available from the Run menu and the toolbar.



Note: If the Coverage drop-down toolbar button is not visible in your current workbench perspective, open the Customize Perspective... dialog and enable the Coverage command group on the Commands tab.

Currently the following launch types are supported:



8

Let's look at command line

First lets make sure we clean out from before:
> mvn clean

```
mcohen>mvn clean
[INFO] Scanning for projects...
[INFO] -----< org.jacoco:org.jacoco.examples.maven >-----
[INFO] Building JaCoCo Maven plug-in example with Offline Instrumentation 0.8.7
[INFO] -----[ jar ]-----
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ org.jacoco.examples.maven ---
[INFO] Deleting /Users/mbcohen/Teaching/COMS 417 S22/exercises/jacoco-0.8.7/examples/triangle/target
[INFO] BUILD SUCCESS
[INFO] Total time: 0.209 s
[INFO] Finished at: 2022-01-25T02:59:38-06:00
[INFO]
```

9

Lets look at it

Now we can run our tests
> mvn test

Note – you will get failures if a junit test fails

```
-----
TESTS
-----
Running TriangleTypeTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.106 sec

Results :

Tests run: 11, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.747 s
[INFO] Finished at: 2022-01-25T03:00:16-06:00
[INFO]
mcohen>
```

10

Lets look at it

Next we need to replace the instrumented code
> mvn jacoco:restore-instrumented-classes

```
mcohen>mvn jacoco:restore-instrumented-classes
[INFO] Scanning for projects...
[INFO] -----< org.jacoco:org.jacoco.examples.maven >-----
[INFO] Building JaCoCo Maven plug-in example with Offline Instrumentation 0.8.7
[INFO] -----[ jar ]-----
[INFO] --- jacoco-maven-plugin:0.8.7:restore-instrumented-classes (default-cli) @ org.jacoco.examples.maven ---
[INFO] BUILD SUCCESS
[INFO] Total time: 0.351 s
[INFO] Finished at: 2022-01-25T03:00:42-06:00
[INFO]
mcohen>
```

11

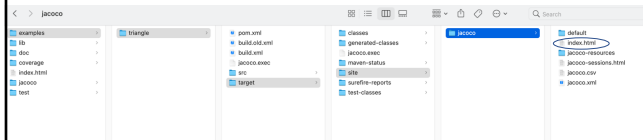
Lets look at it

Now we can make our reports
> mvn jacoco:report

```
cohen>mvn jacoco:report
[INFO] Scanning for projects...
[INFO] -----< org.jacoco:org.jacoco.examples.maven >-----
[INFO] Building JaCoCo Maven plug-in example with Offline Instrumentation 0.8.7
[INFO] -----[ jar ]-----
[INFO] --- jacoco-maven-plugin:0.8.7:report (default-cli) @ org.jacoco.examples.maven ---
[INFO] Loading execution data file /Users/mbcohen/Teaching/COMS 417 S22/exercises/jacoco-0.8.7/examples/triangle/target/jacoco.exec
[INFO] Analyzed bundle 'JaCoCo Maven plug-in example with Offline Instrumentation' with 2 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 0.440 s
[INFO] Finished at: 2022-01-25T03:01:03-06:00
[INFO]
cohen>
```

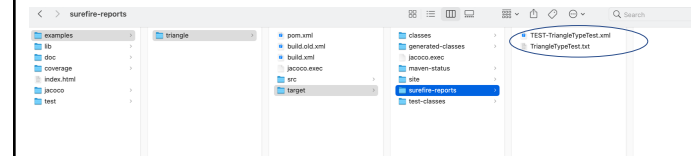
12

Where do we find coverage reports?



13

Where do we find test reports?



14

A Calculator Program

```
import java.io.*;
import java.util.*;

public class calc {
    public int sum(int a, int b) {
        return a + b;
    }
    public int minus(int a, int b) {
        return a - b;
    }
    public int divide(int a, int b) {
        return a / b;
    }
    public int multiply(int a, int b) {
        return a * b;
    }
    public int greater(int a, int b) {
        if (a > b)
            return a;
        else
            return b;
    }
    // only do when mod is positive
    public int mod(int a, int modno) {
        if (modno > 0) {
            return a % modno;
        }
        else
            return -1;
    }
}
```

Cut/paste fault

15

```
import junit.framework.Assert;
import org.junit.Test;

public class calcTest {

    @Test
    public void testSum() {
        calc calculator = new calc();
        Assert.assertEquals(4, calculator.sum(2, 2));
    }

    @Test
    public void testMinus() {
        calc calculator = new calc();
        Assert.assertEquals(0, calculator.minus(2, 2));
    }

    @Test
    public void testDivide() {
        calc calculator = new calc();
        Assert.assertEquals(2, calculator.divide(6, 3));
    }

    @Test(expected = ArithmeticException.class)
    public void testDivideWillThrowExceptionWhenDivideOnZero() {
        calc calculator = new calc();
        calculator.divide(6, 0);
    }
}
```

16

No Fault Found

```

TESTS
-----
Running calcTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.079 sec
Results :

Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.002 s
[INFO] Finished at: 2025-01-27T22:48:46-06:00
[INFO] -----

```

17

Report

calc.java

```

1. import java.io.*;
2. import java.util.*;
3.
4. public class calc{
5.     public int sum(int a, int b){
6.         return a+b;
7.     }
8.     public int minus(int a, int b){
9.         return a-b;
10.    }
11.    public int divide(int a, int b){
12.        return a/b;
13.    }
14.    public int multipl(int a, int b){
15.        return a*b;
16.    }
17.    public int greater(int a, int b){
18.        if(a>b)
19.            return a;
20.        else
21.            return b;
22.    }
23.    // only do when mod is positive
24.    public int mod(int a, int modno){
25.        if(modno>0){
26.            return a%modno;
27.        }
28.        else
29.            return -1;
30.    }
31. }
32.

```

Method not covered

18

JaCoCo

- Creates an “instrumented” directory where it keeps the modified class files

19

Example: Triangle Program

JaCoCo Maven plug-in example with Offline Instrumentation

JaCoCo Maven plug-in example with Offline Instrumentation

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cov.	Missed	Lines	Missed	Methods	Missed	Classes
default	16 of 93	82%	12 of 22	45%	10	14	4	12	1	3	0	2
Total	16 of 93	82%	12 of 22	45%	10	14	4	12	1	3	0	2

20

Example: Triangle Program

```

// Introduction to Software Testing
// Authors: Paul Ammann & Jeff Offutt
// Modified by M.Cohen for class exercise
// Chapter 6;
// See TriangleTypeTest for JUnit tests

public class TriangleType
{
    /**
     * @param s1, s2, s3: sides of the putative triangle
     * @return enum describing type of triangle
     */
    public static Triangle triangle (int s1, int s2, int s3)
    {
        // Reject non-positive sides
        if (s1 <= 0 || s2 <= 0 || s3 <= 0)
            return Triangle.INVALID;

        // Check triangle inequality
        if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
            return Triangle.INVALID;

        // Identify equilateral triangles
        if ((s1 == s2) && (s2 == s3))
            return Triangle.EQUILATERAL;

        // Identify isosceles triangles
        // original program line is below (correct version)
        // if ((s1 == s2) || (s2 == s3) || (s1 == s3))

        // fault introduced below by M.Cohen
        if ((s1 == s3) || (s2 == s3) || (s1 == s3))
            return Triangle.ISOSCELES;
    }
}

```

21

Running

From:

`jacoco-0.8.14/examples/triangle`

- mvn clean
- mvn test (no failures yet)
- mvn jacoco:restore-instrumented-classes
- Mvn jacoco:report

22

Reports

JaCoCo Maven plug-in example with Offline Instrumentation

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cnty	Missed	Lines	Missed	Methods	Missed	Classes
default	82%		45%		10	14	4	12	1	3	0	2
Total	16 of 93	82%	12 of 22	45%	10	14	4	12	1	3	0	2

JaCoCo Maven plug-in example with Offline Instrumentation > default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cnty	Missed	Lines	Missed	Methods	Missed	Classes
TriangleType	67%		45%		10	13	4	10	1	2	0	1
Triangle	100%		n/a		0	1	0	2	0	1	0	1
Total	16 of 93	82%	12 of 22	45%	10	14	4	12	1	3	0	2

23

Reports

```

1. //Introduction to Software Testing
2. //Authors: Paul Ammann & Jeff Offutt
3. //modified by M.Cohen for class exercise
4. //Chapter 6;
5. // See TriangleTypeTest for JUnit tests
6.
7.
8.
9.
10.
11.
12.
13.     * @param s1, s2, s3: sides of the putative triangle
14.     * @return enum describing type of triangle
15.     */
16.     public static Triangle triangle (int s1, int s2, int s3)
17.     {
18.         // Reject non-positive sides
19.         if (s1 <= 0 || s2 <= 0 || s3 <= 0)
20.             return Triangle.INVALID;
21.
22.         // Check triangle inequality
23.         if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
24.             return Triangle.INVALID;
25.
26.         // Identify equilateral triangles
27.         if ((s1 == s2) && (s2 == s3))
28.             return Triangle.EQUILATERAL;
29.
30.         // Identify isosceles triangles
31.         // original program line is below (correct version)
32.         // if ((s1 == s2) || (s2 == s3) || (s1 == s3))
33.
34.         //fault introduced below by M.Cohen
35.         if ((s1 == s3) || (s2 == s3) || (s1 == s3))
36.             return Triangle.ISOSCELES;
37.
38.         return Triangle.SCALENE;
39.     }
40.

```

24

Reports

Source Code Annotation

Line coverage and branch coverage of the active coverage session is also directly displayed in the Java source editors. This works for Java source files contained in the project as well as source code attached to binary libraries.

```

public boolean addAll(int index, Collection c) {
    if(c.isEmpty()) {
        return false;
    } else if(_size == index || _size == 0) {
        return addAll(c);
    } else {
        Listable succ = getListableAt(index);
        Listable pred = (null == succ) ? null : succ.prev();
        Iterator it = c.iterator();
        while(it.hasNext()) {
            pred = insertListable(pred, it.next());
        }
        return true;
    }
}

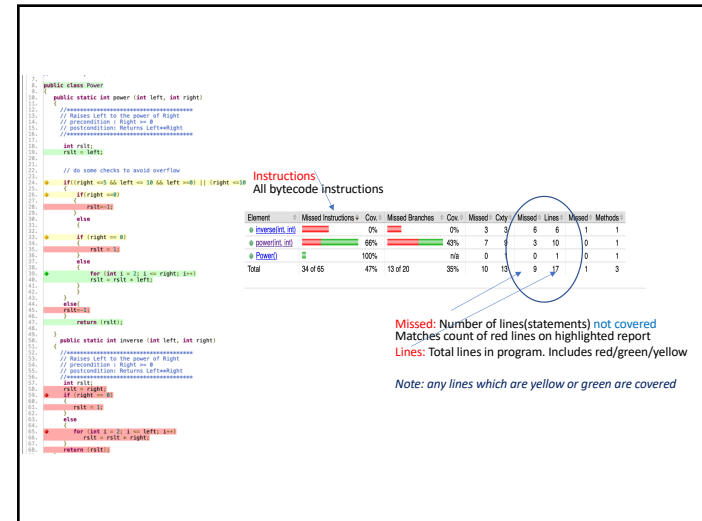
```

Annotations in the code indicate coverage status: green for fully covered lines, yellow for partly covered lines (some instructions or branches missed), and red for lines that have not been executed at all.

Source lines containing executable code get the following color code:

- green for fully covered lines,
- yellow for partly covered lines (some instructions or branches missed) and
- red for lines that have not been executed at all.

25



26

No Fault

- And Low coverage
- Let's add tests
- Live Demo

27

We can Look at the Coverage...

28

Now we Iterate Again

29